

Resenha — The Hexagonal (Ports & Adapters) Architecture

Autor do Artigo: Alistair Cockburn

Autor da Resenha: Felipe Luiz Parreiras Lima

Introdução

O artigo *The Hexagonal (Ports & Adapters) Architecture*, publicado em 2005 por Alistair Cockburn, apresenta uma proposta arquitetural voltada para a separação entre regras de negócio e dependências externas de software. O autor identifica um problema recorrente em sistemas corporativos: a lógica principal da aplicação acaba misturada com interface gráfica, banco de dados, APIs externas e outras tecnologias, dificultando manutenção, testes e evolução do sistema.

Como solução, Cockburn propõe uma arquitetura em que o núcleo da aplicação permanece isolado, comunicando-se com o mundo externo por meio de portas (*ports*) e adaptadores (*adapters*). Essa ideia tornou-se uma das bases modernas para sistemas desacoplados e altamente testáveis.

Ideia central

A principal tese do artigo é que a aplicação deve ser desenvolvida de dentro para fora. Ou seja, primeiro constrói-se o domínio e os casos de uso centrais; depois conectam-se interfaces externas por adaptadores específicos.

Na arquitetura hexagonal:

- **Núcleo interno:** contém regras de negócio e casos de uso.
- **Ports (Portas):** contratos/interfaces que definem como o sistema conversa com o exterior.
- **Adapters (Adaptadores):** implementações concretas que ligam tecnologias reais às portas.

Exemplos de adaptadores:

- Interface web ou mobile
- Banco de dados SQL ou NoSQL
- APIs REST
- Filas de mensageria
- Testes automatizados
- Mock repositories em memória

O formato hexagonal é apenas visual. O número seis não possui significado técnico; o hexágono foi escolhido para demonstrar que podem existir múltiplas conexões externas, fugindo do modelo tradicional em camadas lineares.

Outro ponto importante defendido pelo autor é a facilidade de testes automatizados. Como o núcleo não depende de interface ou banco real, pode ser testado isoladamente com mocks e simulações.

Aplicação em projetos reais

Na engenharia de software atual, essa arquitetura é extremamente útil em sistemas corporativos e produtos digitais escaláveis.

Exemplos práticos:

1. Sistemas bancários

Regras de pagamento, transferência e saldo ficam no núcleo. Banco de dados, gateway PIX e interface web entram como adaptadores.

2. E-commerce

O caso de uso “finalizar compra” permanece independente de Stripe, PayPal, Oracle ou React.

3. Microsserviços

Cada serviço pode possuir domínio interno isolado e múltiplas entradas: REST, eventos Kafka ou jobs agendados.

4. Projetos em crescimento

Permite trocar tecnologias sem reescrever regras de negócio. Exemplo: migrar de MySQL para PostgreSQL ou de Angular para React sem afetar o domínio.

Para times de desenvolvimento, isso reduz acoplamento, facilita manutenção e melhora cobertura de testes.

Conclusão

O artigo de Alistair Cockburn tornou-se uma referência importante da arquitetura de software moderna. Sua proposta resolve um problema clássico: dependência excessiva entre negócio e tecnologia.

A Arquitetura Hexagonal continua atual porque atende demandas contemporâneas como microsserviços, testes automatizados, integração contínua e evolução constante de sistemas. O principal aprendizado do texto é que tecnologias mudam rapidamente, mas regras de negócio tendem a permanecer. Por isso, o núcleo da aplicação deve ser protegido e independente.

Trata-se de um modelo altamente recomendado para projetos que exigem qualidade, escalabilidade e longevidade de código.